Lecture 2 Divide-and-Conquer

Divide-and-Conquer Strategy

- Break a problem into subproblems
- Recursively solve subproblems
- Appropriately combine their answers

Peak Finder

One-dimensional Version

One number in an array is a peak if and only if it is larger than or equal to its neighbor(s).
(if including equals, a peak always exists)

Strategy:

- If `a[n/2] < a[n/2 - 1]`, then only look at left half to look for peak
- Else if `a[n/2] < a[n/2 + 1]`, then only look at right half to look for peak
- Else `n/2` position is a peak.

Complexity:

$$T(n) = T\left(\frac{n}{2}\right) + \Theta(1) = \Theta(1) + \cdots + \Theta(1)(\log_2(n) \text{ times}) = \Theta(\log_2(n))$$

where $\Theta(1)$ is to compare `a[n/2]` to its neighbor(s).

Two-dimensional Version

Two-dimensional Version

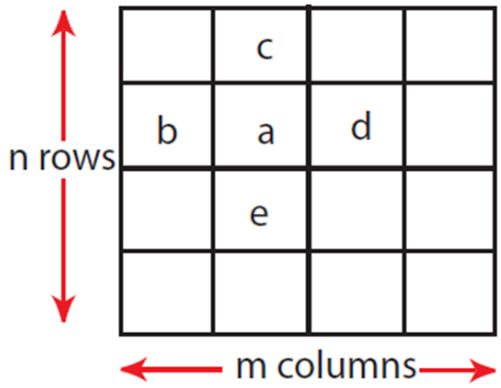


Figure 4: Greedy Ascent Algorithm: $\Theta(nm)$ complexity, $\Theta(n^2)$ algorithm if $m = n$

a is a 2D-peak iff $a \geq b, a \geq d, a \geq c, a \geq e$

Strategy:

- Pick middle column $j = m/2$
- Find global maximum on column j at (i, j)
- Compare $(i, j-1), (i, j), (i, j+1)$
- Pick left half columns if $(i, j-1) > (i, j)$
- Else if $(i, j+1) > (i, j)$ pick right half columns
- Else (i, j) is a 2D-peak

Complexity:
Let $T(n, m)$ denotes the complexity to solve the problem with n rows and m columns.

$$T(n, m) = T(n, m/2) + \Theta(n) = \Theta(n) + \cdots + \Theta(n)(\log m \text{ times}) = \Theta(n \log m)$$

where $\Theta(n)$ is to find global maximum on a column. Since the matrix is symmetric, we can also divide by rows when n and m are extremely different.
If $m \ \ n$, then take middle column; if $n \ \ m$, then take middle row. Thus finally

$$T(n, m) = O(min\{m, n\} \log(ma\{m, n\}))$$

Solving Recurrences

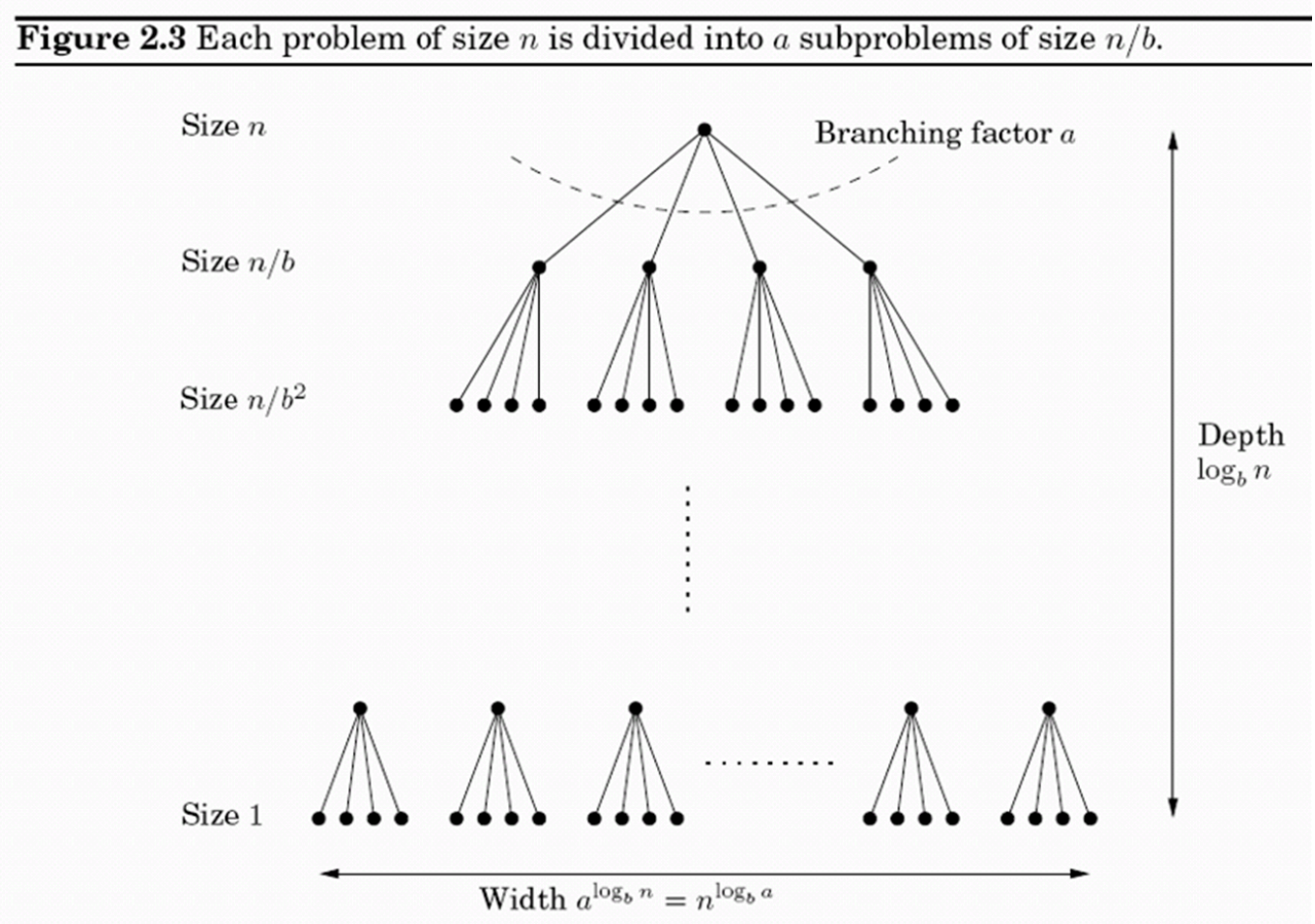
Master Theorem

Divide-and-conquer algorithms tackle a problem of size n by recursively solving a subproblems of size n/b and then combine these answers in $O(n)$ time. $O(n)$ is the polynomial time for all

other efforts except for solving subproblems.

Master Theorem:
If $T(n) = aT(n/b) + O(n)$ for constants $a > 0, b > 1, \geq 0$, then

$$T(n) = O(n), if \ > \log_b a O(n \log_b n), if \ = \log_b a O(n^{\log_b a}), if \ < \log_b a$$



Proof:

- assume n is the power of b
- total work at th level is $a \cdot O\left(\left(\frac{n}{b}\right)\right) = O(n) \cdot \left(\frac{a}{b}\right)$
- the ratio $\frac{a}{b}$ determines the first level(less than one) or the last level(large than one) is dominating. If it is exactly one, it equals the sum of all work and each level has the same work.

Substitution Method

- Guess the form of the solution

- Verify by induction
- Solve for constraints of constants

Recursion-Tree Method

solve the sum of total cost by the recursion tree.